

VisionInspect AI

Manufacturing Defect Detection & Quality Inspection System

Milestone 1 - Technical Documentation

Project Title: VisionInspect AI - Manufacturing Defect Detection & Quality Inspection System

Submitted By: Vinay Kumar Mandalapu

Milestone: 1 - Platform Foundation, Authentication, RBAC, Image Pipeline & Dataset Preparation

Date: September 2026

Version: 1.0

Table of Contents

1. [Executive Summary](#)
2. [Problem Statement](#)
3. [Project Objectives](#)
4. [Scope - Milestone 1 vs Milestone 2](#)
5. [System Architecture](#)
6. [Technology Stack](#)
7. [User Roles & Permissions](#)
8. [User Registration Flow](#)
9. [Authentication System](#)
10. [Role-Based Access Control \(RBAC\)](#)
11. [Image Upload Workflow](#)
12. [Image Storage Design](#)
13. [Image Retrieval & Display](#)
14. [Factory Supervisor Workflow](#)
15. [Database Design](#)
16. [API Design & Endpoint Reference](#)
17. [Security Implementation](#)
18. [Error Handling Strategy](#)
19. [Frontend Architecture](#)
20. [MVTec Anomaly Detection Dataset](#)
21. [Exploratory Data Analysis \(EDA\)](#)
22. [Dataset Loader Implementation](#)
23. [Ground-Truth Masks](#)
24. [UML Diagrams](#)
25. [Testing Strategy & Results](#)
26. [Project Structure](#)
27. [Setup & Deployment](#)
28. [Troubleshooting](#)
29. [Known Limitations](#)
30. [Milestone 2 Transition Plan](#)
31. [End-to-End Flow Summary](#)
32. [Glossary](#)

1. Executive Summary

VisionInspect AI is a full-stack web application designed for manufacturing quality inspection. The system provides a structured workflow where Quality Engineers upload product images for inspection, and Factory Supervisors review those images to approve or reject them based on visual quality standards.

Milestone 1 establishes the complete platform foundation: user authentication with JWT, role-based access control for two distinct user roles, a validated image upload pipeline with secure file storage, a supervisor review workflow, and preparation of the MVTec Anomaly Detection dataset for future AI-based defect detection.

The backend is built with FastAPI and PostgreSQL, the frontend with React 19 and Vite, and the dataset pipeline uses Python with pandas and Pillow. All components are functional and tested with a 20-test integration suite.

2. Problem Statement

Manufacturing environments rely heavily on visual inspection to identify product defects. Traditional manual inspection suffers from:

- **Human fatigue** - inspectors miss defects during long shifts
- **Inconsistency** - different inspectors apply different quality standards
- **No audit trail** - paper-based or ad-hoc inspection processes lack traceability
- **Slow feedback loops** - defects discovered late in the pipeline increase scrap costs

VisionInspect AI addresses these problems by digitizing the inspection workflow. In Milestone 1, the system provides:

- Centralized image upload and storage for inspection images
- Structured review workflow with supervisor approvals
- Complete audit trail with timestamps, reviewer identity, and decision records
- Role separation between Quality Engineers (uploaders) and Factory Supervisors (reviewers)

Milestone 2 will add AI-powered defect detection using convolutional neural networks trained on the MVTec AD dataset.

3. Project Objectives

Milestone 1 Objectives (Implemented)

#	Objective	Status
1	Build a full-stack web application with FastAPI backend and React frontend	Done
2	Implement JWT-based authentication with secure password hashing	Done
3	Implement RBAC with Quality Engineer and Factory Supervisor roles	Done
4	Build a validated image upload pipeline (type, size, content checks)	Done
5	Implement secure file storage with unique filename generation	Done

6	Build supervisor review workflow (approve/reject with notes)	Done
7	Design and seed the PostgreSQL database schema	Done
8	Prepare the MVTec AD dataset and perform EDA	Done
9	Build a dataset loader for training pipeline integration	Done
10	Write integration tests covering all major workflows	Done

Milestone 2 Objectives (Planned)

#	Objective	Status
1	Train a CNN model on MVTec AD for automated defect detection	Planned
2	Integrate model inference into the upload pipeline	Planned
3	Display AI predictions (defect class, confidence, severity) in the UI	Planned
4	Generate defect heatmaps from ground-truth masks	Planned

4. Scope - Milestone 1 vs Milestone 2

Milestone 1 (Current Release)

- User registration and login with role selection
- Supervisor registration code verification
- JWT token issuance and validation
- Role-based route protection (API and frontend)
- Image upload with multi-layer validation
- Secure file storage on disk
- Image listing, detail view, and binary file download
- Supervisor review queue, approval/rejection with notes
- MVTec AD dataset organization and EDA notebook
- Dataset loader for metadata extraction
- Integration test suite (20 tests)
- PlantUML diagrams for system documentation

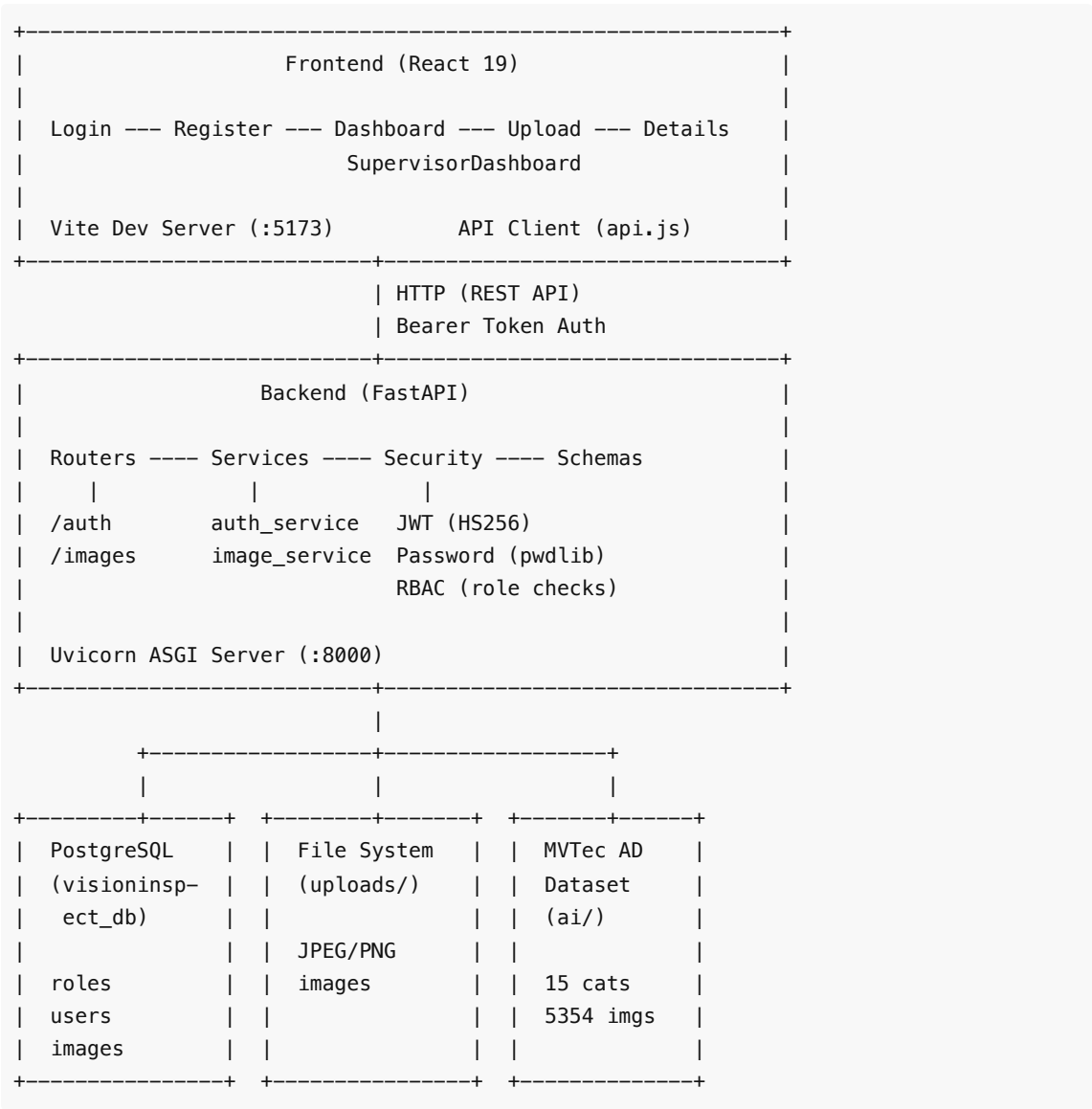
Milestone 2 (Future)

- AI model training on MVTec AD dataset
- Automated defect classification and confidence scoring
- Real-time inference on uploaded images
- Defect severity rating
- Ground-truth mask visualization and heatmap overlays
- Model performance dashboards

AI defect detection is not implemented in Milestone 1. The frontend includes placeholder fields (Defect, Confidence, Severity) that display "-" until the AI pipeline is integrated in Milestone 2.

5. System Architecture

VisionInspect AI follows a three-tier architecture:



Request Flow

- 1. User interacts with the React frontend
- 2. Frontend sends HTTP requests to FastAPI backend with Bearer JWT token
- 3. Backend validates the token, checks role permissions via dependency injection
- 4. Business logic executes in the service layer
- 5. Data persists to PostgreSQL via SQLAlchemy ORM
- 6. Uploaded files store to the uploads/ directory on disk

6. Technology Stack

Backend

Component	Technology	Version	Purpose
-----------	------------	---------	---------

Framework	FastAPI	Latest	Async REST API framework
Server	Uvicorn	Latest	ASGI server
ORM	SQLAlchemy	Latest	Database abstraction layer
Database	PostgreSQL	16	Relational data storage
DB Driver	psycopg2-binary	Latest	PostgreSQL adapter for Python
Auth Tokens	python-jose	Latest	JWT creation and verification
Password Hashing	pwdlib	Latest	Argon2/bcrypt password hashing
Validation	Pydantic	v2	Request/response validation
Configuration	pydantic-settings	Latest	Environment variable management
Image Validation	Pillow (PIL)	Latest	Image format verification
File Upload	python-multipart	Latest	Multipart form data parsing

Frontend

Component	Technology	Version	Purpose
UI Library	React	19.1.0	Component-based UI
Routing	React Router	7.6.1	Client-side navigation
Build Tool	Vite	6.3.5	Dev server and bundler
Linting	ESLint	9.25.0	Code quality enforcement
Fonts	Inter, JetBrains Mono	-	Typography (Google Fonts)

Data Science / AI Preparation

Component	Technology	Purpose
Dataset	MVTec AD	Industrial anomaly detection benchmark
Loader	Python (pathlib)	Dataset metadata extraction
EDA	Jupyter Notebook	Exploratory data analysis
Data Processing	pandas	Tabular data manipulation
Visualization	matplotlib	Charts and plots
Image Processing	Pillow	Image metadata extraction

7. User Roles & Permissions

The system defines two roles, seeded into the `roles` table during database initialization:

Role ID	Role Name	Description
1	Quality Engineer	Uploads inspection images, views all images, views image details
2	Factory Supervisor	Reviews images (approve/reject), accesses supervisor review queue, views all images

Permission Matrix

Action	Quality Engineer	Factory Supervisor
Register account	Yes	Yes (requires access code)
Login	Yes	Yes
View profile (/auth/me)	Yes	Yes
Upload images	Yes	No (403)
View image list	Yes	Yes
View image details	Yes	Yes
Download image file	Yes	Yes
Access review queue	No (403)	Yes
Approve/reject images	No (403)	Yes

8. User Registration Flow

Quality Engineer Registration

1. User navigates to `/register`
2. Selects "Quality Engineer" role card
3. Fills in name, email, password, confirm password
4. Client-side validation checks:
 - o Name is non-empty
 - o Email format is valid
 - o Password meets strength requirements (≥ 8 chars, uppercase, lowercase, digit, special character)
 - o Confirm password matches
5. `POST /auth/register` with `role_id: 1`
6. Backend validates via Pydantic schema (name strip, role_id check, password complexity)
7. Service layer checks email uniqueness in database
8. Password is hashed using `pwdlib.PasswordHash.recommended()`
9. User record created in PostgreSQL
10. Redirect to login page with success message

Factory Supervisor Registration

Steps 1-4 are identical, plus:

5. User selects "Factory Supervisor" role card
6. An additional "Supervisor Access Code" field appears (masked `type="password"`)
7. `POST /auth/register` with `role_id: 2` and `supervisor_registration_code`
8. Backend validates the code against the server-side configured value (loaded from `.env` via `pydantic-settings`)
9. If the code is invalid, registration fails with `400 Bad Request`
10. If valid, user is created with `role_id: 2`

The supervisor access code is never exposed in frontend code, API responses, or error messages. It is configured server-side via the `SUPERVISOR_REGISTRATION_CODE` environment variable.

Password Strength Requirements

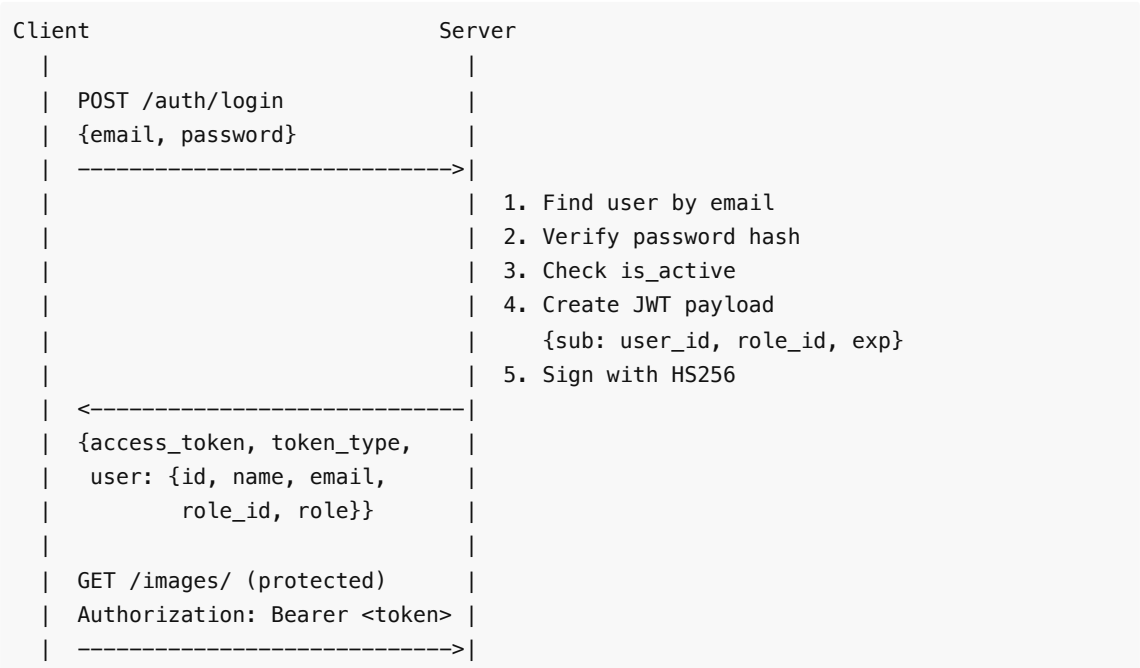
Rule	Implementation
Minimum 8 characters	<code>len(password) < 8</code>
At least one uppercase letter	<code>any(char.isupper() ...)</code>
At least one lowercase letter	<code>any(char.islower() ...)</code>
At least one digit	<code>any(char.isdigit() ...)</code>
At least one special character	<code>any(not char.isalnum() ...)</code>

Validation is enforced at two layers:

- **Frontend:** JavaScript validation before form submission
- **Backend:** Pydantic `@field_validator` on the `RegisterRequest` schema

9. Authentication System

JWT Token Flow



HTTP Request

- > HTTPBearer extracts token
- > `get_current_user` decodes JWT, returns `{user_id, role_id}`
- > `require_role` / `require_any_role` checks permissions
- > Route handler executes

This chain is fully declarative - each endpoint simply declares its required role via `Depends()`, and FastAPI handles the rest.

11. Image Upload Workflow

Image upload is restricted to Quality Engineers (`role_id = 1`).

Validation Pipeline

The upload endpoint applies three sequential validation checks before persisting:

Step	Validation	Method	Failure
1	Content type	<code>file.content_type in {"image/jpeg", "image/png"}</code>	400 Bad Request
2	File size	Stream file in 1 MB chunks, sum total $\leq$ 5 MB	400 Bad Request
3	Image content	<code>PIL.Image.open(file).verify()</code> + format check	400 Bad Request

Upload Sequence

1. Quality Engineer selects a file via drag-and-drop or file picker
2. Frontend validates file type (JPEG/PNG) and size ($\leq$ 5 MB) before sending
3. `POST /images/upload` with `multipart/form-data`
4. Backend runs the three-step validation pipeline
5. Unique filename generated: `{user_id}_{YYYYMMDD_HHMMSS}_{uuid8}.{ext}`
6. File written to `uploads/` directory in 1 MB chunks
7. Database record created with metadata (original filename, stored filename, storage path, uploader ID)
8. On database failure: saved file is deleted from disk, transaction is rolled back
9. Response returns serialized image metadata

Why Three Layers of Validation?

- **Content-Type check** catches obviously wrong files (PDFs, ZIPs renamed to .jpg)
- **Size check** prevents resource exhaustion from large uploads
- **PIL verify** catches corrupted images and files with spoofed MIME types (e.g., a shell script renamed to `image.png` with a forged Content-Type header)

12. Image Storage Design

Directory Structure

```
backend/  
└─ uploads/  
   └─ 1_20260901_143522_a1b2c3d4.jpeg
```

```
|— 1_20260901_144101_e5f6g7h8.png
|— ...
```

Filename Convention

Format: {user_id}_{YYYYMMDD_HHMMSS}_{uuid4_first_8_chars}.{extension}

Component	Purpose
user_id	Links file to uploader without DB query
YYYYMMDD_HHMMSS	Human-readable upload timestamp
uuid4[:8]	Guarantees uniqueness, prevents collisions
.{ext}	Preserves original file extension for MIME detection

Design Decisions

Decision	Rationale
Store on local filesystem	Simple, no cloud dependency for development/MVP
Use generated filenames	Prevents path traversal, filename collisions, and special character issues
Store original filename in DB	Preserves user's filename for display and download headers
Create uploads/ directory automatically	<code>save_image()</code> calls <code>UPLOAD_DIR.mkdir(parents=True, exist_ok=True)</code>
Stream writes in 1 MB chunks	Prevents loading entire file into memory

13. Image Retrieval & Display

Image Listing (GET /images/)

Returns all images sorted by `uploaded_at DESC`. Available to both roles. The response includes serialized image objects with:

- `id`, `original_filename`
- `uploaded_by` (nested object: {`id`, `name`})
- `uploaded_at`, `inspection_status`
- `supervisor_decision`, `supervisor_notes`
- `reviewed_by` (nested object or null), `reviewed_at`

Internal fields (`storage_path` , `stored_filename`) are excluded from the API response.

Image Detail (GET /images/{image_id})

Returns a single image's full metadata. Used by the `ImageDetails` page.

Image File Download (GET /images/{image_id}/file)

Returns the actual image binary via `FileResponse` :

- Sets `media_type` based on file extension (`.jpeg` -> `image/jpeg` , `.png` -> `image/png`)
- Sets `Content-Disposition` with the original filename
- Verifies the physical file exists on disk (404 if missing)
- Requires authentication (Bearer token)

Frontend Thumbnail Loading

The Dashboard components fetch image thumbnails as blob URLs using `getImageBlobUrl(imageId)` . This function:

1. Calls `GET /images/{id}/file` with the Bearer token
2. Converts the response to a `Blob`
3. Creates an object URL via `URL.createObjectURL(blob)`
4. Returns the URL for use in `<img src={...}>`

Thumbnails are lazily loaded for the first 10 images to avoid overwhelming the browser with concurrent requests.

14. Factory Supervisor Workflow

Review Queue

Factory Supervisors access the review queue at `/supervisor/dashboard` . The `GET /images/supervisor/review-queue` endpoint returns all images (pending and reviewed) for the supervisor's queue.

Review Actions

Supervisors can take two actions on each image:

Action	decision value	Notes required?
Approve	"approved"	Optional
Reject	"rejected"	Required

Review Process

1. Supervisor views the review queue with filter tabs: All, Pending, Approved, Rejected
2. Each image card shows: thumbnail, filename, uploader, timestamp, status badge
3. Supervisor enters optional notes in the text field
4. Clicks "Approve" or "Reject"
5. `POST /images/{image_id}/review` with `{decision, notes}`
6. Backend updates: `inspection_status = "reviewed"` , `supervisor_decision` , `supervisor_notes` , `reviewed_by = user_id` , `reviewed_at = datetime.now(UTC)`
7. Success feedback displayed, queue refreshes

Review Data Model

The `ImageReviewRequest` schema uses `Literal["approved", "rejected"]` to restrict the `decision` field to exactly two valid values. Pydantic rejects any other value with a 422 Validation Error.

15. Database Design

Entity-Relationship Diagram

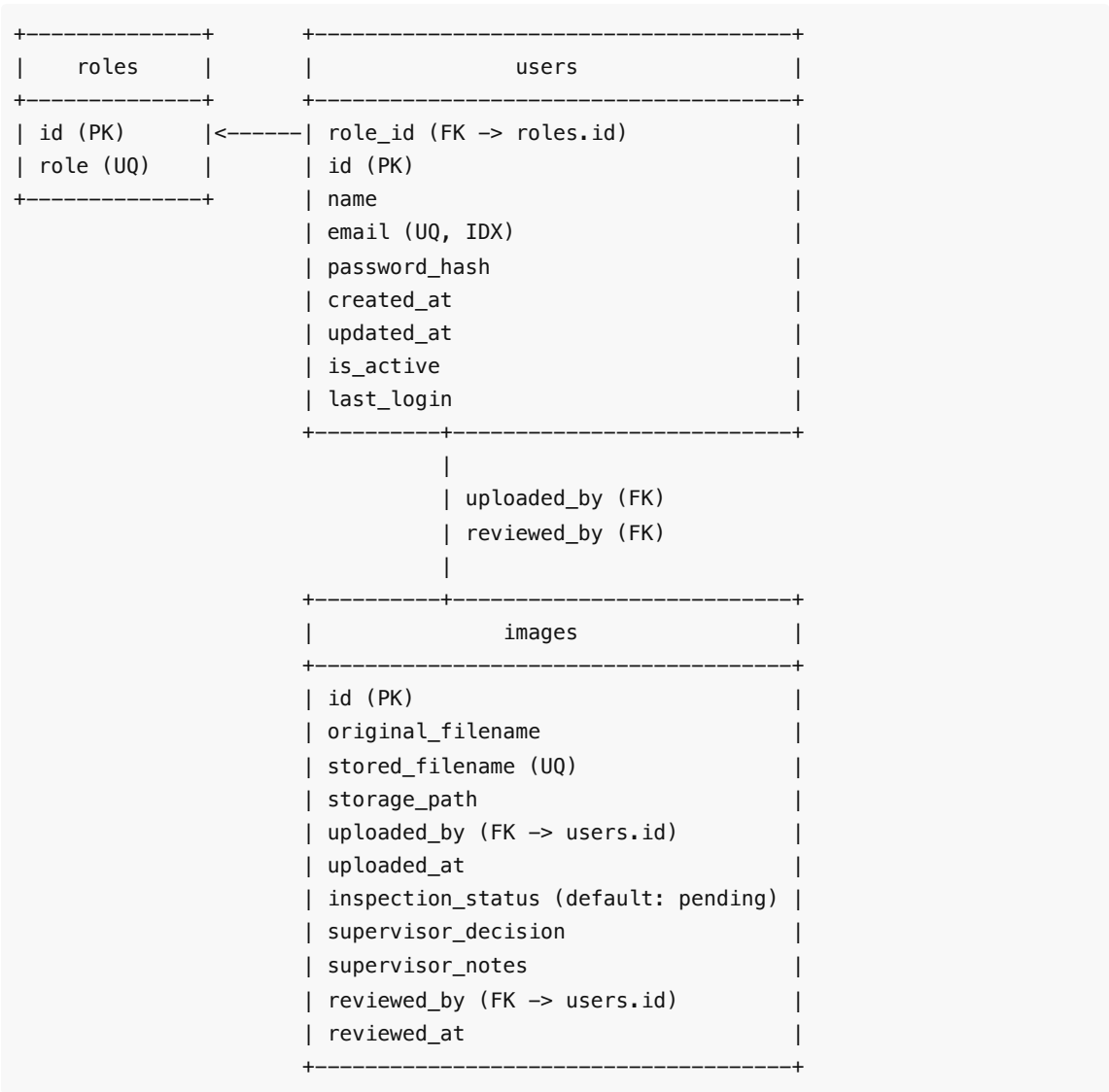


Table: roles

Column	Type	Constraints
id	Integer	PRIMARY KEY, INDEXED
role	String	UNIQUE, NOT NULL

Seeded values: {1: "Quality Engineer", 2: "Factory Supervisor"}

Table: users

Column	Type	Constraints
id	Integer	PRIMARY KEY, INDEXED
name	String	NOT NULL

email	String	NOT NULL, UNIQUE, INDEXED
password_hash	String	NOT NULL
role_id	Integer	FOREIGN KEY -> roles.id
created_at	DateTime (TZ)	DEFAULT now()
updated_at	DateTime (TZ)	ON UPDATE now()
is_active	Boolean	DEFAULT True
last_login	DateTime (TZ)	NULLABLE

Table: images

Column	Type	Constraints
id	Integer	PRIMARY KEY, INDEXED
original_filename	String	NOT NULL
stored_filename	String	NOT NULL, UNIQUE
storage_path	String	NOT NULL
uploaded_by	Integer	FK -> users.id, NOT NULL
uploaded_at	DateTime (TZ)	DEFAULT now(), NOT NULL
inspection_status	String	NOT NULL, DEFAULT "pending"
supervisor_decision	String	NULLABLE
supervisor_notes	String	NULLABLE
reviewed_by	Integer	FK -> users.id, NULLABLE
reviewed_at	DateTime (TZ)	NULLABLE

ORM Relationships

```
# Image model
user = relationship("User", foreign_keys=[uploaded_by])
reviewer = relationship("User", foreign_keys=[reviewed_by])
```

The `Image` model has two foreign keys to `users.id`, disambiguated via `foreign_keys` to establish:

- `image.user` -> the Quality Engineer who uploaded the image
- `image.reviewer` -> the Factory Supervisor who reviewed it (nullable)

16. API Design & Endpoint Reference

Base URL: `http://127.0.0.1:8000`

Authentication Endpoints (/auth)

Method	Path	Auth	Role	Description
POST	/auth/register	None	-	Register a new user
POST	/auth/login	None	-	Authenticate and receive JWT
GET	/auth/me	Bearer	Any	Get current user profile
GET	/auth/supervisor-test	Bearer	Supervisor	RBAC smoke test endpoint

Image Endpoints (/images)

Method	Path	Auth	Role	Description
POST	/images/upload	Bearer	QE only	Upload an image file
GET	/images/	Bearer	QE, Supervisor	List all images
GET	/images/supervisor/review-queue	Bearer	Supervisor only	Get supervisor review queue
GET	/images/{image_id}	Bearer	QE, Supervisor	Get image details
POST	/images/{image_id}/review	Bearer	Supervisor only	Submit review decision
GET	/images/{image_id}/file	Bearer	QE, Supervisor	Download image binary

Utility Endpoints

Method	Path	Auth	Description
GET	/	None	Health check

Request/Response Examples

Register (Quality Engineer):

```
POST /auth/register
Content-Type: application/json

{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "Str0ng@Pass",
  "role_id": 1
}
```

Response 200:

```
{
  "user_id": 1,
  "name": "John Doe",
  "email": "john@example.com",
  "role_id": 1,
  "role": "Quality Engineer"
}
```

Login:

```
POST /auth/login
Content-Type: application/json
```

```
{
  "email": "john@example.com",
  "password": "Str0ng@Pass"
}
```

Response 200:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer",
  "user": {
    "user_id": 1,
    "name": "John Doe",
    "email": "john@example.com",
    "role_id": 1,
    "role": "Quality Engineer"
  }
}
```

Upload Image:

```
POST /images/upload
Authorization: Bearer <token>
Content-Type: multipart/form-data
```

```
file: <binary image data>
```

Response 200:

```
{
  "id": 1,
  "original_filename": "product_shot.jpeg",
  "uploaded_by": {"id": 1, "name": "John Doe"},
  "uploaded_at": "2026-09-01T14:35:22+00:00",
  "inspection_status": "pending",
  "supervisor_decision": null,
  ...
}
```

Review Image:

```
POST /images/1/review
Authorization: Bearer <supervisor_token>
Content-Type: application/json

{
  "decision": "approved",
  "notes": "Product meets quality standards"
}

Response 200:
{
  "id": 1,
  "inspection_status": "reviewed",
  "supervisor_decision": "approved",
  "supervisor_notes": "Product meets quality standards",
  "reviewed_by": {"id": 2, "name": "Supervisor"},
  "reviewed_at": "2026-09-01T15:10:00+00:00",
  ...
}
```

17. Security Implementation

Password Security

Mechanism	Implementation
Hashing library	pwdlib with PasswordHash.recommended()
Hash algorithm	Argon2id (recommended default)
Plain text storage	Never - only hashed values stored
Verification	hasher.verify(plain, hashed) - constant-time comparison

JWT Security

Mechanism	Implementation
Signing algorithm	HMAC-SHA256 (HS256)
Secret key	Server-side only, not exposed to clients
Token expiration	30 minutes
Payload	sub (user_id), role_id, exp (expiration)
Frontend handling	401 -> clear localStorage -> redirect to login

File Upload Security

Threat	Mitigation
Arbitrary file upload	Content-Type whitelist (image/jpeg, image/png)

Oversized files	5 MB limit, validated via streaming (1 MB chunks)
Spoofed MIME types	PIL binary verification (<code>Image.open().verify()</code>)
Path traversal	Generated filenames with UUIDs, no user input in path
Filename collision	UUID component guarantees uniqueness
Orphaned files	Transaction rollback + disk cleanup on DB failure

CORS Configuration

```
allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"]
allow_credentials=True
allow_methods=["*"]
allow_headers=["*"]
```

Restricted to the Vite development server origin. In production, this should be updated to the deployed frontend URL.

Supervisor Registration Code

The supervisor access code is:

- Loaded from the `SUPERVISOR_REGISTRATION_CODE` environment variable via `pydantic-settings`
- Configured in the backend `.env` file
- Never exposed in frontend code, API responses, or error messages
- The registration form uses `type="password"` with a masked placeholder

18. Error Handling Strategy

Backend Error Handling

Scenario	HTTP Status	Detail Message
Invalid registration data	422	Pydantic validation errors (array)
Duplicate email	400	"Email already exists"
Invalid role_id	400	"Selected role is not supported."
Invalid supervisor code	400	"A valid supervisor registration code is required"
Wrong credentials	401	"Invalid email or password"
Inactive account	401	"This account is inactive"
No/invalid JWT	401	"Invalid or expired token"
Wrong role for endpoint	403	"You do not have permission to perform this action"
Image/resource not found	404	"Image not found"

File missing on disk	404	"Image file not found on disk"
Database error on upload	500	"Failed to save image record"
Invalid file type	400	"Only JPEG and PNG images are accepted"
File too large	400	"File size exceeds 5MB limit"
Corrupt image	400	"Uploaded file is not a valid image"

Frontend Error Handling

The API client (`api.js`) defines custom error classes:

Error Class	Triggered By	Frontend Behavior
<code>AuthError</code>	401 response	Clear tokens, redirect to login
<code>ForbiddenError</code>	403 response	Show access denied message
<code>NotFoundError</code>	404 response	Show not found message
<code>ApiError</code>	Other errors	Show error detail

The `apiFetch` function wraps `fetch()` and automatically:

- 1. Attaches the Bearer token from `localStorage`
- 2. Parses error responses and throws the appropriate error class
- 3. Handles Pydantic validation error arrays via `formatErrorMessage()`

19. Frontend Architecture

Component Hierarchy

<code><App></code>		
<code> -- <Login /></code>	Route: <code>/login</code>	
<code> -- <Register /></code>	Route: <code>/register</code>	
<code> -- <Dashboard /></code>	Route: <code>/dashboard</code>	(QE view)
<code> -- <SupervisorDashboard /></code>	Route: <code>/supervisor/dashboard</code>	(Supervisor view)
<code> -- <Upload /></code>	Route: <code>/upload</code>	
<code> -- <ImageDetails /></code>	Route: <code>/images/:imageId</code>	

All pages except Login and Register include the `<Navbar />` component.

Page Descriptions

Page	Route	Role Access	Key Features
Login	<code>/login</code>	Public	Email/password form, role-based redirect on success

Register	/register	Public	Role selection cards, password validation, supervisor code field
Dashboard	/dashboard	QE	KPI cards (total, pending, reviewed), image table with thumbnails
SupervisorDashboard	/supervisor/dashboard	Supervisor	KPI cards (total, pending, reviewed, approved, rejected), filter tabs, inline review
Upload	/upload	QE	Drag-and-drop dropzone, file preview, upload progress
ImageDetails	/images/:imageId	Both	Image preview, metadata panel, review panel

Navigation & Routing

- Login redirects: QE -> /dashboard , Supervisor -> /supervisor/dashboard
- Dashboard auto-redirects: QE accessing supervisor route -> /dashboard , Supervisor accessing QE route -> /supervisor/dashboard
- Upload page shows 403 screen for supervisors
- Unknown routes redirect to /login

State Management

The application uses React component-level state (`useState`) - no global state library. Authentication state is persisted in `localStorage` :

Key	Value
access_token	JWT string
user	JSON-serialized user profile

Design System

The frontend uses a dark industrial theme with CSS custom properties:

Token	Value	Purpose
--obsidian	#0a0e17	Page background
--carbon	#111827	Card/panel background
--steel	#1e293b	Secondary surfaces
--cyan-primary	#0284c7	Primary accent
--cyan-glow	#38bdf8	Hover/focus accent
--status-success	#22c55e	Approved/success states
--status-danger	#ef4444	Rejected/error states

--status-warning	#f59e0b	Pending/warning states
------------------	---------	------------------------

20. MVTec Anomaly Detection Dataset

Overview

The MVTec Anomaly Detection (MVTec AD) dataset is a widely used benchmark for unsupervised anomaly detection in industrial inspection scenarios. It was created by MVTec Software GmbH and published as a research dataset.

The dataset is stored at `ai/dataset/` within the project and organized into 15 product categories.

Dataset Statistics

Metric	Value
Total images	5,354
Training images	3,629
Test images	1,725
Categories	15
Image formats	PNG
Training data	Defect-free ("good") images only
Test data	Both "good" and defective images
Ground-truth masks	Binary masks for defective test images

Category Distribution

Category	Total Images	Rank
hazelnut	501	1
screw	480	2
pill	434	3
carpet	397	4
zipper	391	5
cable	374	6
leather	369	7
capsule	351	8
tile	347	9
grid	342	10

metal_nut	335	11
wood	326	12
transistor	313	13
bottle	292	14
toothbrush	102	15

Directory Structure (per category)

```
category/
|-- train/
|   |-- good/           <- defect-free training images
|-- test/
|   |-- good/           <- defect-free test images
|   |-- defect_type/    <- defective test images (e.g., broken, scratch)
|-- ground_truth/
|   |-- defect_type/    <- binary masks showing defect locations
```

Image Resolutions (sampled)

Resolution	Count (sample of 100)
900 x 900	42
1024 x 1024	33
840 x 840	17
700 x 700	8

All images are square. Resolutions vary by category but are consistent within each category.

21. Exploratory Data Analysis (EDA)

The EDA notebook is located at `notebooks/mvtec_eda.ipynb` .

Analysis Performed

- 1. **Dataset loading** using the `MVTecLoader` class
- 2. **Category distribution** - bar chart showing image count per category
- 3. **Train/test split analysis** - breakdown of training vs. test images per category
- 4. **Defect type analysis** - enumeration of defect sub-types per category
- 5. **Image resolution analysis** - sampling image dimensions to determine size distribution
- 6. **Defective vs. normal distribution** - ratio of good vs. defective images in the test set

Key Findings

- **Imbalanced categories:** hazelnut (501) has ~5x more images than toothbrush (102)
- **Training data is entirely normal:** All training images are labeled "good" (defect-free), consistent with the unsupervised anomaly detection paradigm

- **Defect diversity:** Each category has multiple defect sub-types (e.g., bottle has "broken_large", "broken_small", "contamination")
- **Square images:** All sampled images are square, with resolutions ranging from 700x700 to 1024x1024
- **Ground-truth masks available:** Binary segmentation masks exist for all defective test images

22. Dataset Loader Implementation

The dataset loader is implemented at `ai/dataset/mvtec_loader.py` :

```
class MVTecLoader:
    def __init__(self, dataset_root):
        self.dataset_root = Path(dataset_root)

    def get_categories(self):
        return sorted(
            folder.name
            for folder in self.dataset_root.iterdir()
            if folder.is_dir()
        )

    def load_category(self, category):
        # Returns list of record dicts for a single category
        # Each record: {category, split, defect_type, image_path, mask_path}
```

Features

Feature	Implementation
Category discovery	Auto-discovers subdirectories in the dataset root
Sorted output	Categories sorted alphabetically for deterministic ordering
Train/test split	Iterates both <code>train/</code> and <code>test/</code> subdirectories
Defect type tracking	Uses subdirectory names as defect labels
Mask linkage	For defective test images, resolves the corresponding mask file at <code>ground_truth/{defect_type}/{stem}_mask.png</code>
Missing mask handling	Checks <code>possible_mask.exists()</code> before setting <code>mask_path</code>

Output Schema

Each record returned by `load_category()` is a dictionary:

Field	Type	Description
<code>category</code>	<code>str</code>	Product category name (e.g., "bottle")

split	str	"train" or "test"
defect_type	str	Defect label (e.g., "good", "broken_large", "scratch")
image_path	str	Absolute path to the image file
mask_path	str or None	Absolute path to the mask file (only for defective test images)

23. Ground-Truth Masks

What Are Ground-Truth Masks?

Ground-truth masks are binary segmentation images that mark the exact pixel locations of defects in test images. Each mask is a grayscale PNG where:

- **White pixels (255)** -> defect region
- **Black pixels (0)** -> normal region

Naming Convention

For a test image named `000.png` in defect type `broken_large` :

- Image path: `test/broken_large/000.png`
- Mask path: `ground_truth/broken_large/000_mask.png`

Purpose in Milestone 2

Ground-truth masks will be used in Milestone 2 for:

1. **Model evaluation** - computing pixel-level metrics (IoU, Dice score) for defect localization
2. **Heatmap generation** - overlaying defect regions on the original image for visualization
3. **Training supervision** - if the model architecture supports supervised or semi-supervised learning

24. UML Diagrams

The following PlantUML source files are maintained in the project for system documentation. These diagrams model the system architecture, workflows, and data structures.

Diagram	File	Description
Use Case Diagram	Use_Case_Diagram.puml	Actor-system interaction overview
Class Diagram	Class_Diagram.puml	ORM model structure and relationships
Database ER Diagram	Database_ER_Diagram.puml	Entity-relationship schema
Component Architecture	Component_Architecture.puml	System component layout
Deployment Diagram	Deployment_Diagram.puml	Infrastructure and deployment topology
Activity Diagram (M1)	Activity_Diagram_milestone1.puml	Milestone 1 workflow
Auth/RBAC Sequence	Authentication_RBAC_Sequence.puml	Login and authorization flow

QE Upload Sequence	QE_Upload_Sequence.puml	Image upload interaction
Supervisor Review Sequence	Supervisor_Review_Sequence.puml	Supervisor review interaction
Inspection State Diagram	Inspection_State_Diagram.puml	Image inspection lifecycle states

These PlantUML files can be rendered using any PlantUML-compatible tool (VS Code extension, PlantUML server, IntelliJ, etc.).

25. Testing Strategy & Results

Test Framework

Tests use Python's built-in `unittest` module with a custom ASGI test client (`tests/asgi_client.py`). The custom client was built to avoid dependency on `httpx` (required by Starlette's `TestClient`), which was not installed.

Custom ASGI Test Client

The `ASGITestClient` class directly invokes the FastAPI ASGI app in-process:

- Supports `GET` , `POST` with JSON bodies
- Supports `multipart/form-data` for file uploads (custom multipart encoder)
- Runs async ASGI calls via `asyncio.run()`
- Returns `ASGIResponse` objects with `.status_code` , `.json()` , `.text` , `.content`

Test Suite: `test_milestone1.py`

#	Test Name	Validates
1	<code>test_01_root_health_check</code>	<code>GET /</code> returns 200
2	<code>test_02_register_quality_engineer_success</code>	QE registration works, password not in response
3	<code>test_03_register_supervisor_without_code_fails</code>	Supervisor registration requires code
4	<code>test_04_register_supervisor_with_invalid_code_fails</code>	Wrong code is rejected
5	<code>test_05_register_supervisor_with_valid_code_success</code>	Correct code succeeds
6	<code>test_06_register_duplicate_email_fails</code>	Duplicate email returns 400
7	<code>test_07_register_invalid_role_id_fails</code>	Invalid <code>role_id</code> returns 400/422
8	<code>test_08_register_weak_password_fails</code>	Weak passwords return 422
9	<code>test_09_login_quality_engineer</code>	QE login returns token
10	<code>test_10_login_factory_supervisor</code>	Supervisor login returns token
11	<code>test_12_get_current_user_me</code>	<code>/auth/me</code> returns authenticated user profile

12	test_13_qe_upload_image_success	QE can upload PNG image
13	test_14_supervisor_cannot_upload_image	Supervisor upload returns 403
14	test_15_both_roles_can_view_images	Both roles can list images
15	test_16_supervisor_can_access_review_queue	Supervisor accesses review queue
16	test_17_qe_cannot_access_supervisor_review_queue	QE gets 403 on review queue
17	test_18_supervisor_can_review_image	Supervisor can approve/reject
18	test_19_qe_cannot_review_image	QE gets 403 on review
19	test_20_protected_image_file_download	Authenticated file download works
20	test_21_unauthenticated_requests_fail	No/invalid token returns 401/403

Results

All 20 tests pass.

```
-----
Ran 20 tests in X.XXXs

OK
```

Test Coverage Areas

Area	Tests
Authentication	8 (register x 6, login x 2)
Authorization (RBAC)	5 (upload denied, review queue denied, review denied, both view, unauthenticated)
Image Upload	2 (QE success, supervisor denied)
Image Retrieval	2 (both roles list, file download)
Supervisor Review	2 (supervisor review, QE denied)
Health Check	1

26. Project Structure

```
VisionInspect_AI/
|-- README.md
|-- .gitignore
|
|-- backend/
|   |-- .env           <- Environment variables
|   |-- uploads/       <- Uploaded image files
|   |-- myvenv/        <- Python virtual environment
```

```

| |-- app/
| | |-- main.py                <- FastAPI app entry point
| | |-- core/
| | | |-- config.py           <- Settings (pydantic-settings)
| | | |-- database/
| | | | |-- base.py           <- SQLAlchemy Base
| | | | |-- connection.py     <- Engine, SessionLocal, get_db
| | | | |-- init_db.py        <- Table creation, role seeding
| | | |-- models/
| | | | |-- __init__.py       <- Model exports
| | | | |-- role.py           <- Role model
| | | | |-- user.py           <- User model
| | | | |-- image.py          <- Image model
| | | |-- schemas/
| | | | |-- auth_schema.py    <- RegisterRequest, LoginRequest
| | | | |-- image_schema.py   <- ImageReviewRequest
| | | |-- security/
| | | | |-- jwt.py            <- Token creation/verification
| | | | |-- password.py       <- Hash/verify passwords
| | | | |-- dependencies.py    <- get_current_user dependency
| | | | |-- authorization.py   <- require_role, require_any_role
| | | | |-- roles.py          <- Role constants, supervisor code
| | | |-- services/
| | | | |-- auth_service.py    <- register_user, login_user
| | | | |-- image_service.py   <- Upload, validation, review logic
| | | |-- routers/
| | | | |-- auth.py           <- /auth endpoints
| | | | |-- image.py          <- /images endpoints
| |-- tests/
| | |-- asgi_client.py         <- Custom ASGI test client
| | |-- test_milestone1.py     <- 20 integration tests
|
|-- frontend/
| |-- index.html              <- HTML entry point
| |-- package.json            <- Dependencies & scripts
| |-- vite.config.js          <- Vite configuration
| |-- eslint.config.js        <- ESLint configuration
| |-- src/
| | |-- main.jsx              <- React entry point
| | |-- App.jsx               <- Router & routes
| | |-- App.css               <- Component styles
| | |-- index.css             <- Global styles & tokens
| | |-- pages/
| | | |-- Login.jsx           <- Login page
| | | |-- Register.jsx        <- Registration page
| | | |-- Dashboard.jsx       <- QE dashboard
| | | |-- SupervisorDashboard.jsx <- Supervisor dashboard
| | | |-- Upload.jsx          <- Image upload page
| | | |-- ImageDetails.jsx     <- Image detail view
| | |-- components/
| | | |-- Navbar.jsx          <- Navigation bar
| | |-- services/

```

```
|
|         |-- api.js                <-- API client & error classes
|
|-- ai/
|   |-- dataset/
|       |-- mvtec_loader.py         <-- Dataset loader class
|       |-- (15 category directories)
|
|-- mvtec_anomaly_detection/        <-- Raw dataset storage
|
|-- notebooks/
|   |-- mvtec_eda.ipynb             <-- EDA Jupyter notebook
|
|-- docs/
|   |-- (PlantUML diagram files)
```

27. Setup & Deployment

Prerequisites

- Python 3.9+
- Node.js 18+
- PostgreSQL 16
- Git

Backend Setup

```
# 1. Create and activate virtual environment
cd backend
python3 -m venv myvenv
source myvenv/bin/activate

# 2. Install dependencies
pip install fastapi uvicorn sqlalchemy psycpg2-binary \
    python-jose pwdlib pydantic pydantic-settings \
    python-multipart pillow

# 3. Create PostgreSQL database
createdb visioninspect_db

# 4. Configure environment variables
# Create backend/.env with:
# SUPERVISOR_REGISTRATION_CODE=<your_code>

# 5. Initialize database (creates tables, seeds roles)
PYTHONPATH=. python app/database/init_db.py

# 6. Start the server
uvicorn app.main:app --reload
```

Frontend Setup

```
cd frontend

# 1. Install dependencies
npm install

# 2. Start development server
npm run dev
# Opens at http://localhost:5173

# 3. Build for production
npm run build

# 4. Lint check
npm run lint
```

Database Reset (All Records)

To clear all image records and uploaded files:

```
cd backend
PYTHONPATH=. ./myenv/bin/python -c \
    "from app.database.connection import SessionLocal; \
    from app.models.image import Image; \
    db=SessionLocal(); db.query(Image).delete(); \
    db.commit(); db.close()" && rm -rf uploads/*
```

Running Tests

```
cd backend
PYTHONPATH=. ./myenv/bin/python -m pytest tests/test_milestone1.py -v
```

28. Troubleshooting

Issue	Cause	Solution
ModuleNotFoundError: app.xxx	Missing PYTHONPATH	Run with PYTHONPATH=. python ...
CORS errors in browser	Backend not running or wrong origin	Ensure backend runs on :8000, frontend on :5173
401 on all requests	Expired or missing JWT	Re-login to get a fresh token
Image preview shows stale images	DB records exist but files deleted	Clear both DB records and uploads/ directory
Upload returns 400 (not valid image)	File is not a valid JPEG/PNG	Verify the file opens correctly in an image viewer

Supervisor registration fails	Wrong or missing access code	Verify the SUPERVISOR_REGISTRATION_CODE in .env
psycopg2 connection error	PostgreSQL not running	Start PostgreSQL: brew services start postgresql
Frontend blank page	CSS not loading	Ensure import './App.css' exists in main.jsx
npm run lint warnings	ESLint react-hooks/exhaustive-deps	Include navigate in useEffect dependency arrays

29. Known Limitations

Limitation	Impact	Planned Resolution
No AI defect detection	Images must be manually reviewed	Milestone 2 - CNN model integration
Local file storage	Not suitable for multi-server deployment	Migrate to cloud storage (S3) in production
No pagination	All images loaded at once	Add server-side pagination for large datasets
No password reset flow	Users cannot recover lost passwords	Add email-based password reset
JWT secret in source code	Security risk if code is leaked	Move to environment variable
No HTTPS	Traffic is unencrypted in development	Configure TLS for production deployment
No image compression/thumbnails	Full-size images served for previews	Generate thumbnails on upload
Single-server architecture	No horizontal scaling	Containerize and add load balancer
No audit log table	Review history tied to image record	Add separate audit trail table

30. Milestone 2 Transition Plan

AI Integration Points

The Milestone 1 codebase is designed with Milestone 2 integration in mind:

1. **Image model** already has fields for AI results:

- `inspection_status` - currently set to "pending" on upload, will transition to "inspected" after AI processing

- Future columns: `defect_class` , `confidence_score` , `severity_rating` can be added to the `Image` model

2. **Dataset loader** (`ai/dataset/mvtec_loader.py`) is ready for the training pipeline:

- Loads metadata for all categories
- Links test images to ground-truth masks
- Returns structured records for DataFrame creation

3. **EDA results** from the notebook provide baseline understanding:

- Category distribution guides training data sampling
- Image resolution analysis informs input preprocessing
- Defect type enumeration defines the classification taxonomy

4. **Upload pipeline** will be extended to trigger inference:

- After file save, queue the image for model prediction
- Store prediction results (class, confidence, severity) in the database
- Update `inspection_status` from "pending" to "inspected"

5. **Frontend** has placeholder UI for AI results:

- `ImageDetails.jsx` shows Defect, Confidence, Severity fields (currently "-")
- These will display actual model predictions in Milestone 2

Planned Architecture Changes

```
Current (M1):    Upload -> Save -> Manual Review
Future  (M2):    Upload -> Save -> AI Inference -> Display Results -> Supervisor
Review
```

31. End-to-End Flow Summary

Quality Engineer Flow

1. Register at `/register` with role "Quality Engineer"
2. Login at `/login` -> receive JWT -> redirect to `/dashboard`
3. Navigate to `/upload`
4. Drag-and-drop or select JPEG/PNG image (`<=5MB`)
5. Image is validated (type, size, content) and saved
6. View uploaded image in `/dashboard` table
7. Click "Inspect Record" -> `/images/:id` for full details
8. Wait for supervisor review

Factory Supervisor Flow

1. Register at `/register` with role "Factory Supervisor"
(requires supervisor access code)
2. Login at `/login` -> receive JWT -> redirect to `/supervisor/dashboard`
3. View all images in the review queue
4. Filter by status: All, Pending, Approved, Rejected
5. For each pending image:

- a. View thumbnail and metadata
 - b. Enter review notes (required for rejection)
 - c. Click "Approve" or "Reject"
6. Decision is recorded with timestamp and reviewer identity
7. Quality Engineer sees updated status in their dashboard

Image Lifecycle

pending -> reviewed (approved)

pending -> reviewed (rejected)

32. Glossary

Term	Definition
JWT	JSON Web Token - a compact, self-contained token for securely transmitting information between parties
RBAC	Role-Based Access Control - access decisions based on the roles assigned to users
ASGI	Asynchronous Server Gateway Interface - Python async web server specification
ORM	Object-Relational Mapping - technique for querying databases using object-oriented code
CORS	Cross-Origin Resource Sharing - HTTP mechanism allowing cross-origin requests
MVTec AD	MVTec Anomaly Detection dataset - industrial defect detection benchmark
PIL	Python Imaging Library (Pillow) - image processing library
EDA	Exploratory Data Analysis - initial data investigation and visualization
QE	Quality Engineer - user role that uploads images for inspection
FS	Factory Supervisor - user role that reviews and approves/rejects images
KPI	Key Performance Indicator - metrics displayed on dashboards
UUID	Universally Unique Identifier - 128-bit number for unique identification
Pydantic	Data validation library using Python type annotations
FastAPI	Modern Python web framework for building APIs
SQLAlchemy	Python SQL toolkit and ORM
Vite	Next-generation frontend build tool
Ground-truth mask	Binary image showing exact defect pixel locations
pwdlib	Password hashing library supporting Argon2/bcrypt

Bearer token	HTTP authentication scheme where the token is passed in the Authorization header
---------------------	--

33. Milestone 1 Completion Checklist

#	Deliverable	Status
1	FastAPI backend with Uvicorn	Done
2	React 19 frontend with Vite	Done
3	PostgreSQL database with SQLAlchemy ORM	Done
4	User registration (QE and Supervisor)	Done
5	JWT-based authentication (HS256, 30-min expiry)	Done
6	Password hashing (pwdlib/Argon2)	Done
7	Role-based access control (require_role, require_any_role)	Done
8	Supervisor registration code verification	Done
9	Image upload with 3-layer validation (type, size, content)	Done
10	Secure file storage with UUID-based filenames	Done
11	Image listing and detail retrieval	Done
12	Authenticated binary file download	Done
13	Supervisor review queue	Done
14	Supervisor approve/reject with notes	Done
15	Transaction safety (rollback + file cleanup)	Done
16	QE Dashboard with KPIs and image table	Done
17	Supervisor Dashboard with filter tabs and inline review	Done
18	Responsive dark-theme UI	Done
19	MVTec AD dataset organized (15 categories, 5354 images)	Done
20	EDA notebook with category/split/resolution analysis	Done
21	Dataset loader (MVTecLoader with mask linkage)	Done
22	PlantUML diagrams (10 diagrams)	Done
23	Integration test suite (20 tests, all passing)	Done
24	Frontend lint: 0 errors, 0 warnings	Done
25	Frontend build: clean production build	Done

26	No secrets exposed in frontend or API responses	Done
----	---	------

End of Milestone 1 Documentation